

C# Self Study — From No. 2 to No. 4

Topics: List & ArrayList • dynamic vs var • Anonymous Types & Compiler

2. Built-in Functions in List and ArrayList

List<T> is a generic collection used to store elements of a specific type. The type is specified between < >, such as List<int> or List<string>.

Method / Property	Purpose
Add()	Adds one element to the end of the list.
AddRange()	Adds multiple elements.
Insert()	Adds an element at a specific index.
Remove()	Removes the first occurrence of a specified value.
RemoveAt()	Removes the element at a specific index.
Contains()	Checks whether a value exists in the list.
Clear()	Removes all elements.
Sort()	Sorts the elements.
Reverse()	Reverses the order of the elements.
Count	Returns the number of elements.

Example:

```
List<string> names = new List<string>();

names.Add("Ahmed");
names.Add("Mona");
names.Insert(1, "Omar");

names.Remove("Mona");

Console.WriteLine(names.Count);
```

Assignment note: If the question explicitly asks you to search using a loop, do not use Contains(). Use for or foreach.

```
bool found = false;

for (int i = 0; i < names.Count; i++)
{
    if (names[i] == "Ahmed")
    {
        found = true;
        break;
    }
}
```

Array

An array stores elements of a specific type and has a fixed size after it is created.

```
int[] numbers = { 10, 20, 30 };
```

```
Console.WriteLine(numbers.Length);
```

Common Array operations include `Length`, `Array.Sort()`, `Array.Reverse()`, and `Array.IndexOf()`. When the assignment requires loops, implement the operation with a loop.

ArrayList

`ArrayList` is an older, non-generic collection from `System.Collections`. Its elements are handled as object, so it can contain different types.

```
ArrayList values = new ArrayList();

values.Add(10);
values.Add("Rawan");
values.Add(true);
values.Add(20.5);
```

List<T> vs ArrayList

```
List<int> numbers = new List<int>();

numbers.Add(10);
// numbers.Add("Rawan");    // Error: List<int> expects int

ArrayList values = new ArrayList();

values.Add(10);
values.Add("Rawan");
values.Add(true);
```

Conclusion: `List<T>` is generic and strongly typed, while `ArrayList` is non-generic and can store different types as objects. In modern C#, `List<T>` is generally preferred when the element type is known.

3. What is the role of dynamic in Web Applications, and how is it different from var?

var allows the compiler to infer the variable's type from its assigned value. It is still statically typed.

```
var name = "Rawan";  
var age = 21;
```

The compiler knows that name is a string and age is an int. Assigning a different type later is not allowed.

```
var age = 21;  
  
// age = "Rawan";    // Compile-time error
```

dynamic defers type-related checking for operations on the variable to runtime. The variable can hold values of different types during execution.

```
dynamic value = 10;  
  
value = "Rawan";  
value = true;
```

Example:

```
dynamic value = 10;  
  
Console.WriteLine(value.Length);
```

This can pass compilation because the member access is resolved dynamically, but at runtime it can fail because an int does not have a Length property.

Role of dynamic in web applications

dynamic can be useful when working with data whose structure is not strongly known at compile time, such as some dynamic JSON or external-data scenarios. When the structure is known, strongly typed models are usually clearer and provide compile-time checking.

```
public class User  
{  
    public string Name { get; set; }  
    public string Email { get; set; }  
}  
  
User user = GetUser();
```

Main difference

var	dynamic
Statically typed	Resolved dynamically at runtime
Type inferred at compile time	Less compile-time type checking
Type remains fixed	Can hold values of different types
Many type errors caught earlier	Some errors appear at runtime

4. Anonymous Types and What the Compiler Does

An anonymous type is a type created by the C# compiler without the developer explicitly giving the type a name.

```
var product = new  
{
```

```

    Name = "Laptop",
    Price = 25000
};

```

There are two different concepts here:

1. **var** — tells the compiler to infer the variable's type from the assigned expression.
2. **new { }** — creates an anonymous type.

Conceptually, you can imagine the compiler generating something similar to:

```

class GeneratedAnonymousType
{
    public string Name { get; }
    public int Price { get; }
}

```

This is only a conceptual representation. The compiler-generated type has an internal generated name; you do not normally write or access that generated class by name.

Does the compiler create it and then delete it?

No. The compiler generates the required type as part of compilation. You do not manually create or delete the anonymous type.

The **object** created from that type is a runtime object. When it is no longer reachable through any references, it becomes eligible for garbage collection. The .NET Garbage Collector manages its memory.

Simplified lifecycle:

```

C# Source Code
    |
    v
Compiler
    |
    v
Generated Anonymous Type
    |
    v
Anonymous Object at Runtime
    |
    v
No More References
    |
    v
Eligible for Garbage Collection

```

Important distinction: the compiler generates the type; the Garbage Collector manages the memory of runtime objects when they are no longer referenced. The developer does not write `delete product;`.

Submission-ready answers

1. List and ArrayList: `List<T>` is a generic, strongly typed collection, while `ArrayList` is a non-generic collection that can store different types as objects. `List<T>` is generally preferred in modern C# when the element type is known because it provides better type safety.

2. dynamic vs var: `var` is statically typed and its type is inferred at compile time. `dynamic` defers member and operation checking to runtime, so some errors that would be caught during compilation with `var` can instead occur during execution. `dynamic` can be useful for some data whose structure is not known strongly at compile time.

3. Anonymous types: An anonymous type is generated by the compiler when using `new { }`. The developer does not need to define or delete a separate class. Runtime objects created from the

anonymous type are managed by the .NET Garbage Collector when they are no longer reachable.